

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

1. TITLE

Demonstrate JavaScript Arrays, Functions, and Array Methods

2. AIM

To write and execute a JavaScript program demonstrating 1D, 2D, and multi-dimensional arrays, arrays of objects, various function types (normal, anonymous, arrow, callback, recursive), and ES6 array methods (map, filter, reduce, forEach).

3. OBJECTIVE

The objectives of this experiment are:

- To understand creation, indexing, and iteration of 1D, 2D, and multi-dimensional JavaScript arrays.
 - To work with arrays of objects and iterate using array iteration techniques.
 - To implement normal functions, functions with/without arguments, and anonymous functions.
 - To write concise ES6 arrow functions and short arrow syntax.
 - To pass and execute callback functions inside higher-order functions.
 - To write recursive functions with base termination conditions.
 - To apply modern ES6 array transformation methods (map(), filter(), reduce(), forEach()).
-

4. THEORY

JavaScript is a versatile scripting language that supports dynamic data structures and functional programming paradigms.

Core concepts demonstrated in this program:

- Arrays: Indexed collection of data values (1D, 2D matrix arrays, multi-dimensional, and arrays of objects).

- Functions: Block of reusable statements. JS supports normal declarations, anonymous function expressions, and ES6 arrow functions ((n) => n * n).
- Callback Functions: Functions passed as arguments to execute asynchronously or conditionally.
- Recursive Functions: Functions that invoke themselves until reaching a base condition.
- Array Methods: Higher-order functions like map() (transforms elements), filter() (selects elements), reduce() (accumulates single value), and forEach() (iterates elements).

The basic flow of execution is:

JavaScript Code -> V8 Engine Execution -> Array & Function Computation -> Console Output

5. ALGORITHM / PROCEDURE

1. Create a JavaScript file named exp3.js.
2. Declare a 1D array colors and output to console.
3. Declare a 2D array students storing student name and age pairs; iterate using a for loop.
4. Create a multi-dimensional nested array subjects and print structure.
5. Construct an array of objects books containing book names and prices; iterate using forEach().
6. Define a normal function add(a, b) that returns addition of two numbers.
7. Define a function welcome() without parameters.
8. Define a function multiply(a, b) returning product of arguments.
9. Implement an anonymous function subtract assigned to a variable.
10. Write an arrow function square(n) returning square of n.
11. Write a short arrow function cube = n => n * n * n.
12. Implement a callback function calculate(a, b, operation) passing an addition function.
13. Implement a recursive function factorial(n) with base condition n <= 1.
14. Declare a numbers array [2, 4, 6, 8, 10] and apply map(), filter(), reduce(), and forEach().
15. Execute exp3.js using Node.js (node exp3.js) and verify output.

6. PROGRAM

exp3.js

```
// =====
// JAVASCRIPT PROGRAM: ARRAYS AND FUNCTIONS
// =====

// ----- 1. ONE-DIMENSIONAL ARRAY -----
let colors = ["Red", "Blue", "Green", "Yellow"];
```

```

console.log("1. One-Dimensional Array:");
console.log(colors);

// ----- 2. TWO-DIMENSIONAL ARRAY -----
let students = [
    ["Arjun", 18],
    ["Meena", 19],
    ["Kiran", 18]
];

console.log("\n2. Two-Dimensional Array:");

for (let i = 0; i < students.length; i++) {
    console.log("Name:", students[i][0],
        "Age:", students[i][1]);
}

// ----- 3. MULTI-DIMENSIONAL ARRAY -----
let subjects = [
    [
        ["Maths", 80],
        ["English", 75]
    ],
    [
        ["Science", 85],
        ["Computer", 90]
    ]
];

console.log("\n3. Multi-Dimensional Array:");
console.log(subjects);

// ----- 4. ARRAY OF OBJECTS -----
let books = [
    { name: "JavaScript", price: 500 },
    { name: "Python", price: 400 },
    { name: "HTML", price: 300 }
];

console.log("\n4. Array of Objects:");

books.forEach(function(book) {
    console.log("Book:", book.name,
        "Price:", book.price);
});

// =====
// FUNCTIONS
// =====

// ----- 5. NORMAL FUNCTION -----
function add(a, b) {

```

```

        return a + b;
    }

    console.log("\n5. Normal Function:");
    console.log("Addition =", add(5, 10));

    // ----- 6. FUNCTION WITH NO ARGUMENTS -----
    function welcome() {
        console.log("Welcome to JavaScript!");
    }

    console.log("\n6. Function with No Arguments:");
    welcome();

    // ----- 7. FUNCTION WITH ARGUMENTS -----
    function multiply(a, b) {
        return a * b;
    }

    console.log("\n7. Function with Arguments:");
    console.log("Multiplication =", multiply(4, 5));

    // ----- 8. ANONYMOUS FUNCTION -----
    let subtract = function(a, b) {
        return a - b;
    };

    console.log("\n8. Anonymous Function:");
    console.log("Subtraction =", subtract(15, 6));

    // ----- 9. ARROW FUNCTION -----
    let square = (n) => {
        return n * n;
    };

    console.log("\n9. Arrow Function:");
    console.log("Square =", square(7));

    // ----- 10. SHORT ARROW FUNCTION -----
    let cube = n => n * n * n;

    console.log("\n10. Short Arrow Function:");
    console.log("Cube =", cube(4));

    // ----- 11. CALLBACK FUNCTION -----
    function calculate(a, b, operation) {
        return operation(a, b);
    }

    let answer = calculate(8, 2, function(a, b) {

```

```

        return a + b;
    });

    console.log("\n11. Callback Function:");
    console.log("Answer =", answer);

    // ----- 12. RECURSIVE FUNCTION -----
    function factorial(n) {

        if (n == 0 || n == 1) {
            return 1;
        }

        return n * factorial(n - 1);
    }

    console.log("\n12. Recursive Function:");
    console.log("Factorial of 4 =", factorial(4));

    // =====
    // ARRAY METHODS
    // =====

    let numbers = [2, 4, 6, 8, 10];

    console.log("\n13. Array Methods:");

    // ----- map() -----
    let doubleNumbers = numbers.map(function(num) {
        return num * 2;
    });

    console.log("map() =", doubleNumbers);

    // ----- filter() -----
    let greaterNumbers = numbers.filter(function(num) {
        return num > 5;
    });

    console.log("filter() =", greaterNumbers);

    // ----- reduce() -----
    let total = numbers.reduce(function(sum, num) {
        return sum + num;
    }, 0);

    console.log("reduce() =", total);

    // ----- forEach() -----
    console.log("forEach():");

```

```
numbers.forEach(function(num) {  
    console.log(num);  
});
```

7. CODE EXPLANATION

Code	Explanation
let colors = [...]	Declares a 1-Dimensional string array.
students[i][0]	Accesses elements in a 2-Dimensional matrix array.
books.forEach()	Iterates over each object item inside an array of objects.
function add(a, b)	Defines a standard function accepting two parameters.
let subtract = function	Creates an anonymous function assigned to a variable.
let square = (n) => ...	ES6 arrow function for concise function expression.
calculate(a, b, op)	Higher-order function accepting a callback operation function.
factorial(n)	Recursive function computing factorial using self-invocation.
numbers.map()	Creates a new array by doubling every number element.
numbers.filter()	Filters array elements returning numbers greater than 5.
numbers.reduce()	Accumulates all array values into a single total sum.

8. TEST CASES AND EXECUTION DATA

Test Case	Feature Tested	Test Input	Expected Result	Actual Result	Status
TC-01	2D Array Traversal	students array	Print Name and Age for 3 items	Printed 3 rows correctly	Pass
TC-02	Arrow Functions	square(7), cube(4)	Returns 49 and 64	Returned 49 and 64	Pass

TC-03	Callback Function	calculate(8, 2, add)	Returns calculated sum 10	Returned 10	Pass
TC-04	Recursive Factorial	factorial(4)	Computes 4! = 24	Computed 24	Pass
TC-05	Array Methods	map, filter, reduce	Outputs doubled, filtered, sum	Output matches expected data	Pass

9. OUTPUT

Output 1: Server / Command Line Execution Data (node exp3.js)

```

PS C:\Users\Hari Charan\OneDrive\Desktop\fullstack> node .\record\exp3\exp3.js
1. One-Dimensional Array:
[ 'Red', 'Blue', 'Green', 'Yellow' ]

2. Two-Dimensional Array:
Name: Arjun Age: 18
Name: Meena Age: 19
Name: Kiran Age: 18

3. Multi-Dimensional Array:
[
  [ [ 'Maths', 80 ], [ 'English', 75 ] ],
  [ [ 'Science', 85 ], [ 'Computer', 90 ] ]
]

4. Array of Objects:
Book: JavaScript Price: 500
Book: Python Price: 400
Book: HTML Price: 300

5. Normal Function:
Addition = 15

6. Function with No Arguments:
Welcome to JavaScript!

7. Function with Arguments:
Multiplication = 20

8. Anonymous Function:
Subtraction = 9

9. Arrow Function:
Square = 49

10. Short Arrow Function:
Cube = 64

11. Callback Function:
Answer = 10

12. Recursive Function:
Factorial of 4 = 24

13. Array Methods:
map() = [ 4, 8, 12, 16, 20 ]
filter() = [ 6, 8, 10 ]
reduce() = 30
forEach():
2
4
6
8
10

```

10. RESULT

Thus, a JavaScript program covering 1D/2D arrays, function types, arrow syntax, recursion, and ES6 array methods was executed successfully using Node.js and verified across all test cases.